

Zaawansowane Wzorce Projektowe i Architektura Systemów IT

KOMPENDIUM WIEDZY DLA PROGRAMISTÓW • FORMAT FISZKI BLOKOWEJ

Część I: Paradygmaty i Zasady Projektowe (SOLID)

S - Single Responsibility Principle (Zasada Jednej Odpowiedzialności)

Klasa powinna mieć tylko jeden powód do modyfikacji. Oznacza to, że powinna być odpowiedzialna za dokładnie jedną, wąsko zdefiniowaną funkcjonalność w systemie. Zapobiega powstawaniu tzw. "klas-bogów" (God Objects).

O - Open/Closed Principle (Zasada Otwarte/Zamknięte)

Elementy systemu (klasy, moduły) powinny być otwarte na rozszerzenie, ale zamknięte na modyfikację. Zmianę zachowania realizuje się poprzez dopisywanie nowego kodu (np. dziedziczenie, polimorfizm, implementacja interfejsu), a nie zmianę istniejącego.

L - Liskov Substitution Principle (Zasada Podstawienia Liskov)

Obiekty klas pochodnych muszą być w stanie zastąpić obiekty klas bazowych bez wpływu na poprawność działania programu. Klasa podrzędna nie może łamać kontraktu ani założeń semantycznych klasy nadrzędnej.

I - Interface Segregation Principle (Zasada Segregacji Interfejsów)

Klienci nie powinni być zmuszani do zależności od interfejsów, których nie używają. Lepiej stworzyć wiele mniejszych, dedykowanych interfejsów niż jeden ogólny, przeładowany metodami.

D - Dependency Inversion Principle (Zasada Odwrócenia Zależności)

Moduły wysokopoziomowe nie powinny zależeć od modułów niskopoziomowych. Oba powinny zależeć od abstrakcji. Ponadto abstrakcje nie powinny zależeć od szczegółów, ale szczegóły powinny zależeć od abstrakcji.

Koncepcja: YAGNI (You Aren't Gonna Need It)

Zasada programistyczna mówiąca, że nie należy dodawać funkcjonalności ani pisać kodu dopóki nie stanie się on absolutnie niezbędny. Zapobiega przekomplikowaniu architektury i marnowaniu zasobów na over-engineering.

Część II: Wzorce Kreacyjne i Strukturalne

Wzorzec: Fabryka Abstrakcyjna (Abstract Factory)

Dostarcza interfejs do tworzenia całych rodzin powiązanych lub zależnych od siebie obiektów bez określania ich konkretnych klas. Umożliwia pełną izolację procesu tworzenia obiektów od kodu klienckiego.

Wzorzec: Budowniczy (Builder)

Rozdziela proces konstruowania złożonego obiektu od jego reprezentacji. Dzięki temu ten sam proces konstrukcyjny może tworzyć różne reprezentacje obiektu. Szczególnie przydatny przy obiektach z dużą liczbą parametrów opcjonalnych.

Wzorzec: Singleton

Gwarantuje, że klasa będzie miała tylko jedną instancję w skali całej aplikacji, oraz zapewnia globalny punkt dostępu do tej instancji. Uwaga: często uważany za antywzorzec (Anti-pattern), ponieważ utrudnia testowanie jednostkowe i wprowadza ukryte stany globalne.

Wzorzec: Adapter (Pełnomocnik strukturalny)

Przekształca interfejs klasy na inny interfejs, którego oczekują klienci. Umożliwia współdziałanie klasom, które ze względu na niekompatybilne interfejsy nie mogłyby ze sobą współpracować. Działa jako wrapper.

Wzorzec: Dekorator (Decorator)

Pozwala na dynamiczne dodawanie nowych odpowiedzialności i zachowań do obiektu bez konieczności stosowania dziedziczenia. Polega na opakowaniu oryginalnego obiektu w obiekt dekoratora implementujący ten sam interfejs.

Wzorzec: Fasada (Facade)

Udostępnia jednolity, uproszczony interfejs wysokiego poziomu dla całego podsystemu skomplikowanych klas. Ukrywa wewnętrzne powiązania i złożoność komponentów przed klientem zewnętrznym.

Wzorzec: Proksy (Proxy / Pełnomocnik)

Zapewnia obiekt pośredniczący lub zastępczy (surrogate) dla innego obiektu w celu kontrolowania dostępu do niego. Może służyć do leniwego ładowania (Virtual Proxy), kontroli uprawnień (Protection Proxy) lub logowania operacji.

Część III: Wzorce Behawioralne i Architektura Rozproszona

Wzorzec: Obserwator (Observer)

Definiuje zależność typu jeden-do-wielu pomiędzy obiektami. Kiedy stan jednego obiektu (podmiotu) ulega zmianie, wszyscy jego obserwatorzy są automatycznie powiadamiani i aktualizowani. Fundament architektury sterowanej zdarzeniami.

Wzorzec: Strategia (Strategy)

Definiuje rodzinę wymiennych algorytmów, hermetyzuje każdy z nich i czyni je całkowicie niezależnymi od klienta, który z nich korzysta. Pozwala na zmianę algorytmu w czasie działania programu (Runtime).

Wzorzec: Stan (State)

Pozwala obiektowi na zmianę swojego zachowania w zależności od wewnętrznego stanu, w którym się znajduje. Obiekt sprawia wrażenie, jakby zmienił swoją klasę. Eliminuje skomplikowane i rozbudowane instrukcje warunkowe typu if/else lub switch.

Wzorzec: Metoda Szablonowa (Template Method)

Definiuje szkielet algorytmu w metodzie klasy bazowej, opóźniając implementację niektórych kroków do klas pochodnych. Pozwala podklasom na przededefiniowanie niektórych etapów algorytmu bez zmiany jego ogólnej struktury.

Architektura: CQRS (Command Query Responsibility Segregation)

Wzorzec architektoniczny polegający na całkowitym rozdzieleniu operacji modyfikujących dane (Command) od operacji odczytujących dane (Query). Pozwala na niezależną optymalizację i skalowanie modelu zapisu i odczytu.

Architektura: Event Sourcing (Źródło Zdarzeń)

Podejście architektoniczne, w którym stan aplikacji nie jest przechowywany jako aktualny rekord w bazie, lecz jako sekwencja niezmiennych zdarzeń (events). Stan końcowy odtwarza się poprzez sekwencyjne przetwarzanie historii zdarzeń.

Koncepcja: CAP Theorem (Twierdzenie CAP)

W rozproszonym systemie przechowywania danych możliwe jest jednoczesne zapewnienie tylko dwóch z trzech gwarancji: Spójności (Consistency), Dostępności (Availability) oraz Odporności na podział sieci (Partition Tolerance).

Koncepcja: Eventual Consistency (Spójność Ostateczna)

Model spójności stosowany w systemach rozproszonych. Gwarantuje, że przy braku nowych aktualizacji, wszystkie repliki danych ostatecznie zsynchronizują się i będą zwracać identyczną, najnowszą wartość.